INTEL 8086

Features of 8086 Microprocessor:

- Intel 8086 was launched in 1978.
- It was the first 16-bit microprocessor.
- This microprocessor had major improvement over the execution speed of 8085.
- It is available as 40-pin Dual-Inline-Package (DIP).
- It is available in three versions:
  a. 8086 (5 MHz)

  b. 8086-2 (8 MHz)
  c. 8086-1 (10 MHz)
- It consists of 29,000 transistors.
- It has a 16 line data bus and 20 line address bus.
- It could address up to 1 MB of memory.
- It has more than 20,000 instructions.

PIN DIAGRAM OF 8086

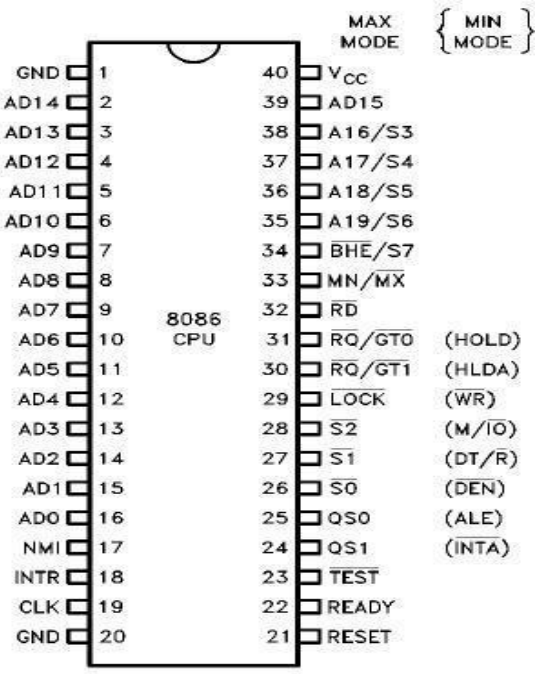


Figure 2.1 Pin diagram

AD_0-AD_{15}

These lines are multiplexed bidirectional address/data bus. During T1, they carry lower order 16-bit address. In the remaining clock cycles,they carry 16-bit data. AD0-AD7 carry lower order byte of data. AD8-AD15 carry higher order byte of data.

A19/S6, A18/S5, A17/S4, A16/S3 [Pin 35-38 (Unidirectional)]

These lines are multiplexed unidirectional address and status bus. During T1, they carry higher order 4-bit address. In the remaining clock cycles, they carry status signals.

BHE / S7 [Pin 34 (Output)]

BHE stands for Bus High Enable. BHE signal is used to indicate the transfer of data over higher order data bus (D8 – D15). 8-bit I/O devices use this signal. It is multiplexed with status pin S7.

RD (Read) [Pin 32 (Output)]

It is a read signal used for read operation. It is an output signal. It is an active low signal.

READY [Pin 22 (Input)]

This is an acknowledgement signal from slower I/O devices or memory. It is an active high signal. When high, it indicates that the device is ready to transfer data. When low, then microprocessor is in wait state.

RESET[Pin 21 (Input)]

It is a system reset. It is an active high signal. When high, microprocessor enters into reset state and terminates the current activity. It must be active for atleast four clock cycles to reset the microprocessor.

INTR [Pin 18 (Input)]

It is an interrupt request signal. It is active high. It is level triggered

NMI[Pin 17 (Input)]

It is a non-maskable interrupt signal. It is an active high. It is an edge triggered interrupt.

TEST[Pin 23 (Input)]

It is used to test the status of math coprocessor 8087. The BUSY pin of 8087 isconnected to this pin of 8086. If low, execution continues else microprocessor is in wait state.

CLK[Pin 19 (Input)]

This clock input provides the basic timing for processor operation. It is symmetric square wave with 33% duty cycle. The range of frequency of different versions is 5 MHz, 8 MHz and 10 MHz.

VCC and VSS[Pin 40 and Pin 20 (Input)]

VCC is power supply signal. +5V DC is supplied through this pin. VSS is ground signal

MN / MX[Pin 33 (Input)]

8086 works in two modes: Minimum Mode, Maximum Mode. If MN/MX is high, it works in minimum mode. If MN/MX is low, it works in maximum mode. Pins 24 to 31 issue two different sets of signals. One set of signals is issued when CPU operates in minimum mode. Other set of signals is issued when CPU operates in maximum mode.

PIN DESCRIPTION FOR MINIMUM MODE

INTA [Pin 24 (Output)]

This is an interrupt acknowledge signal. When microprocessor receives INTR signal, it acknowledges the interrupt by generating this signal. It is an active low signal

ALE [Pin 25 (Output)]

This is an Address LatchEnable signal. It indicates that valid address is available on bus AD0 – AD15. It is an active high signal and remains high during T1 state. It is connected to enable pin of latch 8282.

DEN[Pin 26 (Output)]

This is a Data Enable signal. This signal is used toenable the transceiver 8286. Transceiver is used to separate the data from the address/data bus. It is an active low signal

DT / R[Pin 27 (Output)]

This is a DataTransmit/Receive signal. It decides the direction ofdata flow through the transceiver. When it is high, data is transmitted out. When it is low, data is received in.

M / IO[Pin 28 (Output)]

This signal is issued by the microprocessor to distinguish memory access from I/O access. When it is high, memory is accessed. When it is low, I/O devices are accessed.

WR [Pin 29 (Output)]

It is a Write signal. It is used to write data inmemory or output device depending on the status of M/IO signal. It is an active low signal

HLDA[Pin 30 (Output)]

It is a Hold Acknowledge signal. It is issued after receiving the HOLD signal. It is an active high signal

HOLD[Pin 31 (Input)]

When DMA controller needs to use address/data bus, it sends a request to the CPU through this pin. It is an active high signal. When microprocessor receives HOLD signal, it issues HLDA signal to the DMA controller.

PIN DESCRIPTION FOR MAXIMUM MODE

QS1 and QS0 [Pin 24 and 25 (Output)]

These pins provide the status of instruction queue.

QS1	QS_0	STATUS
0	0	NO OPERATION
0	1	1st byte of opcode from queue
1	0	Empty queue
1	1	Subsequent byte from queue

S0, S1, S2 [Pin 26, 27, 28 (Output)]

These status signals indicate the operation being done by the microprocessor. This information is required by the Bus Controller 8288. Bus controller 8288 generates all memory and I/O control signals.

S_0	S_1	S_2	STATUS
0	0	0	Interrupt Acknowledge
0	0	1	I/O Read
0	1	0	I/O Write

0	1	1	Halt
1	0	0	Opcode Fetch
1	0	1	Memory Read
1	1	0	Memory Write
1	1	1	Passive

LOCK[Pin 29 (Output)]

- This signal indicates that other processors should not ask CPU to relinquish the system bus. When it goes low, all interrupts are masked and HOLD request is not granted. This pin is activated by using LOCK prefix on any instruction.

RQ/GT1 and RQ/GT0 [Pin 30 and 31 (Bi-directional)]

- These are Request/Grant pins. Other processors request the CPU through these lines to release the system bus. After receiving the request, CPU sends acknowledge signal on the same lines. RQ/GT0 has higher priority than RQ/GT1.

ARCHITECTURE OF 8086

The microprocessors functions as the CPU in the stored program model of the digital computer. Its job is to generate all system timing signals and synchronize the transfer of data between memory, I/O, and itself. The microprocessor also has a S/W function. It must recognize, decode, and execute program instructions fetched from the memory unit. This requires an Arithmetic-Logic Unit (ALU) within the CPU to perform arithmetic and logical (AND, OR, NOT, compare, etc) functions.

The 8086 has pipelined architecture. The 8086 CPU is organized as two separate processors, called the Bus Interface Unit (BIU) and the Execution Unit (EU).

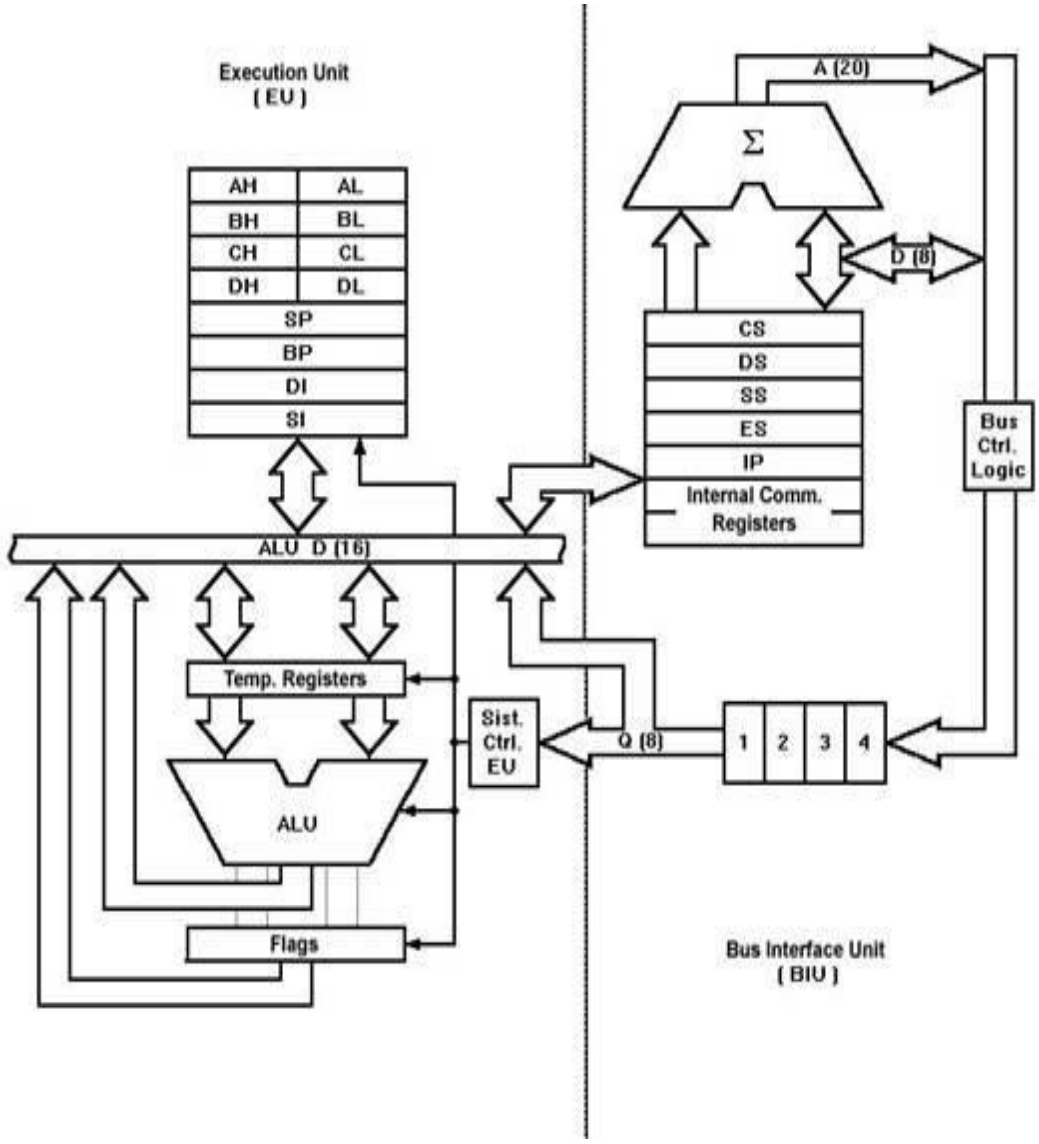


Fig : 2.2Architecture of 8086

BUS INTERFACE UNIT (BIU)

The function of BIU is to:

- Fetch the instruction or data from memory.
- Write the data to memory.
- Write the data to the port.
- Read data from the port.

INSTRUCTION QUEUE

- To increase the execution speed, BIU fetches as many as six instruction bytes ahead to time from memory.
- All six bytes are then held in first in first out 6 byte register called instruction queue.
- Then all bytes have to be given to EU one by one.
- This pre fetching operation of BIU may be in parallel with execution operation of EU, which improves the speed execution of the instruction.

EXECUTION UNIT (EU)

The functions of execution unit are:

- To tell BIU where to fetch the instructions or data from.
- To decode the instructions.
- To execute the instructions.

The EU contains the control circuitry to perform various internal operations. A decoder in EU decodes the instruction fetched memory to generate different internal or external control signals required to perform the operation. EU has 16-bit ALU, which can perform arithmetic and logical operations on 8-bit as well as 16-bit.

GENERAL PURPOSE REGISTERS OF 8086

These registers can be used as 8-bit registers individually or can be used as 16-bit in pair to have AX, BX, CX, and DX.

- AX Register: AX register is also known as accumulator register that stores operands for arithmetic operation like divided, rotate.
- BX Register: This register is mainly used as a base register. It holds the starting base location of a memory region within a data segment.
- CX Register: It is defined as a counter. It is primarily used in loop instruction to store loop counter.
- DX Register: DX register is used to contain I/O port address for I/O instruction.

SEGMENT REGISTERS :

Additional registers called segment registers generate memory address when combined with other in the microprocessor. In 8086 microprocessor, memory is divided into 4 segments as follow:

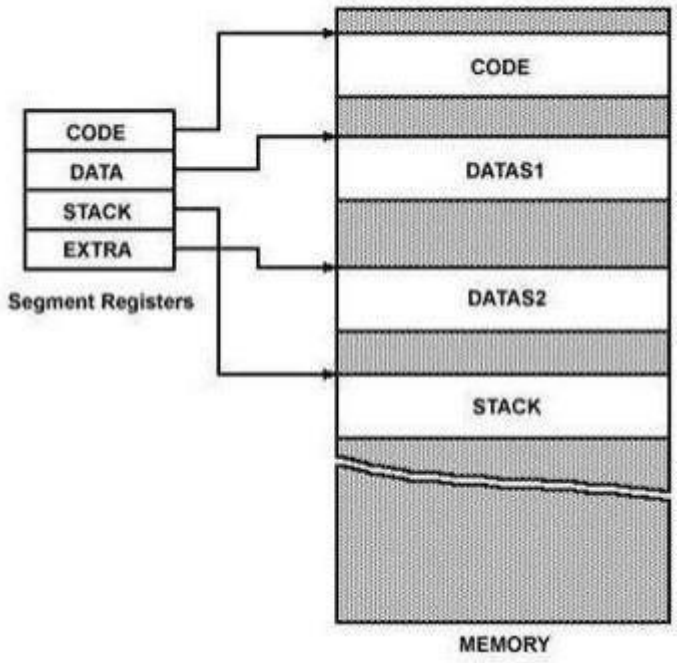


Fig.2.3 Memory Segments of 8086

- Code Segment (CS): The CS register is used for addressing a memory location in the Code Segment of the memory, where the executable program is stored.
- Data Segment (DS): The DS contains most data used by program. Data are accessed in the Data Segment by an offset address or the content of other register that holds the offset address.
- Stack Segment (SS): SS defined the area of memory used for the stack
- Extra Segment (ES): ES is additional data segment that is used by some of the string to hold the destination data.

Flag register in EU is of 16-bit , is shown

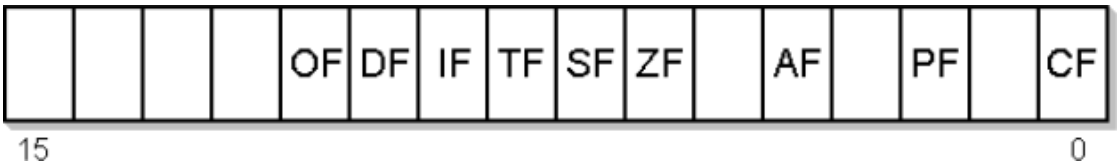


Fig : Flag Register of 8086

Flag Register determines the current state of the processor. They are modified automatically by CPU after mathematical operations, this allows to determine the type of the result, and

to determine conditions to transfer control to other parts of the program. 8086 has 9 flags and they are divided into two categories:

1. Conditional Flags

2. Control Flags

CONDITIONAL FLAGS

Conditional flags represent result of last arithmetic or logical instruction executed. Conditional flags are as follows:

1. Carry Flag (CF): This flag indicates an overflow condition for unsigned integer arithmetic. It is also used in multiple-precision arithmetic.

2. Auxiliary Flag (AF): If an operation performed in ALU generates a carry/barrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), the AF flag is set i.e. carry given by D3 bit to D4 is AF flag. This is not a general-purpose flag, it is used internally by the processor to perform Binary to BCD conversion.

3. Parity Flag (PF): This flag is used to indicate the parity of result. If lower order 8- bits of the result contains even number of 1's, the Parity Flag is set and for odd number of 1's, the Parity Flag is reset.

4. Zero Flag (ZF): It is set; if the result of arithmetic or logical operation is zero else it is reset.

5. Sign Flag (SF): In sign magnitude format the sign of number is indicated by MSB bit. If the result of operation is negative, sign flag is set.

6. Overflow Flag (OF): It occurs when signed numbers are added or subtracted. An OF indicates that the result has exceeded the capacity of machine.

CONTROL FLAGS

Control flags are set or reset deliberately to control the operations of the execution unit. Control flags are as follows:

1. Trap Flag (TP):

    a. It is used for single step control.
    b. It allows user to execute one instruction of a program at a time for debugging.

    c. When trap flag is set, program can be run in single step mode.

2. Interrupt Flag (IF):

    a. It is an interrupt enable/disable flag.
    b. If it is set, the maskable interrupt of 8086 is enabled and if it is reset, the interrupt is disabled.

    c. It can be set by executing instruction sit and can be cleared by executing CLI instruction.

3. Direction Flag (DF):

    a. It is used in string operation.
    b. If it is set, string bytes are accessed from higher memory address to lower memory address.

    c. When it is reset, the string bytes are accessed from lower memory address to higher memory address.

ADDRESSING MODES OF 8086

ADDRESSING MODES FOR REGISTER AND IMMEDIATE DATA

- Register Addressing mode
- Immediate Addressing mode

Addressing modes for memory data

- Register Indirect Addressing mode
- Direct Addressing mode
- Based Addressing mode
- Indexed Addressing mode
- Base Relative Addressing mode
- Base Indexed Addressing mode
- String Addressing Mode

Addressing modes for I/O port

- Direct I/O port Addressing
- Indirect I/O port Addressing

RELATIVE ADDRESSING

- Implied Addressing Mode

REGISTER ADDRESSING MODE

Data transfer using registers is called register addressing mode. Here operand value is present in register. For example

```
MOV
AL,BL
; MOV
AX,B
X;
```

IMMEDIATE ADDRESSING MODE

When data is stored in code segment instead of data segment immediate addressing mode is used. Here operand value is present in the instruction. For example

```
MOV AX, 0A9FH;
```

DIRECT ADDRESSING MODE

When direct memory address is supplied as part of the instruction is called direct addressing mode. Operand offset value with respect to data segment is given in instruction. For example

```
MOV      AX,
[089DH];
ADD      AX,
[0ADH];
```

REGISTER INDIRECT ADDRESSING MODE

Here operand offset is given in a CPU register. Register used are BX, SI (source index), DI (destination index), or BP(base pointer). BP holds offset w.r.t Stack segment, but SI, DI and BX refer to data segment. For example-

```
MOV [BX], AX;
ADD AX, [SI];
```

BASED ADDRESSING MODE

In this mode EA is obtained by adding a displacement (signed 8 bit or unsigned 16 bit) value to the contents of BX or BP. The segment registers used are DS & SS.

When Memory is accessed, the 20 bit physical address is computed from BX and DS .On the other hand, when the stack is accessed, the 20 bit physical address is computed

from BP and SS.

eg:- MOV AL,START[BX] or MOV AL,[BX+START]

Where START=02H (8 bit displacement),BX=2000H

Now the 20 bit Physical address is computed from DS and EA

Here the source operand is in based Addressing Mode .EA is obtained by adding the value of START and [BX].The 20 bit physical address is produced from DS and EA. The 8 bit content of this memory location is moved to AL register.

INDEXED ADDRESSING MODE

In this mode,the EA is calculated by adding the unsigned 16 bit or signed extended 8 bit displacement and the contents of SI or DI.

eg:- MOV BH,START[SI]

Moves the contents of the 20 bit address computed from the displacement START, SI and DS into BH register. The 8 bit displacement is provided by the programmer using the assembler pseudo instruction such as EQU. For 16 bit displacement, the EU adds this to SI to determine EA. On the other hand, for 8 bit displacement the EU sign extends it to 16 bits and then adds to SI for determining EA.

BASE RELATIVE ADDRESSING MODE

Operand offset given by a sum of a value held either in BP, or BX and a constant offset specified as an operand. For example
MOV
AX,[BP+1];
JMP [BX+1];

BASE INDEXED ADDRESSING MODE

Here operand offset is given by sum of either BX or BP with either SI or DI. For example
MOV AX,
[BX+S] JMP
[BP+DI]

STRING ADDRESSING

This mode uses index registers. The string instructions automatically assume SI to point to the first byte or word of the source operand and DI to point to the first byte or word of the destination operand.

The segment register for the source is DS and may be overridden. The segment register for the destination must be ES and cannot be overridden.

The contents of SI and DI are automatically incremented by clearing DF (Direction Flag) to 0 by CLD instruction or automatically decremented by setting DF to 1 by STD instruction.

DIRECT ADDRESSING MODES

Here the port number is a 8 bit immediate operand.This allows fixed access to ports numbered 0 to 255.

eg:- OUT 05H,AL

outputs [AL] to 8 bit port 05H

INDIRECT ADDRESSING MODE

The port number is taken from DX allowing 64K 8 bit ports or 32K 16 bit ports.
eg:- IN AX,DX
If [DX]=5040,Inputs the 8 bit content of port 5040 into AL and 5041 into AH.

RELATIVE ADDRESSING MODE

In this mode, the operand is specified as a signed 8 bit displacement, relative to PC(Program Counter).

eg:- JNC START

Then, if carry=0,PC is loaded with current PC contents plus the 8 bit signed value of START, otherwise the next instruction is executed.

IMPLIED ADDRESSING MODE

Instructions using this mode have no operands.
eg:- CLC
This clears the carry flag to zero.